

Medical Journey Log: A Family Care Tracker
Alex Frear
CST-451 Milestone 3: Final Architectural Plan (Design Phase)
Grand Canyon University
Instructor: Professor Scott Webster
Revision: 1
7/9/2025

Table of Contents

Abstract	3
Final Technical Requirements.....	4
System Functional Requirements.....	4
System Architecture	4
Software Requirements	4
Hardware Requirements.....	4
Design Planning Summary	5
Overview of Design Concepts	6
Main Navigation Flow	6
User Interface Mockups	7
1. Dashboard	7
2. Log Symptom.....	8
3. Appointment Manager	9
4. Journal Entry	10
5. Reports	11
Pseudocode Example: Log Symptom.....	12
Design Priorities.....	12
Detailed Solution Architecture	13
System Architecture Overview	13
Object and Data Element Definitions	13
Class: FamilyMember	14
Class: SymptomEntry.....	14
Class: Appointment	15
Class: JournalEntry	15
Class: ReportService	16
Entity-Relationship Diagram / Database Schema	17
Planned Tables:	17
Table Definitions	18
Sequence Diagram	19
Final UI Screen Specifications.....	20
Dashboard	20
Log Symptom.....	20
Appointment Manager	21
Journal Entry	21
Reports	21
Security Considerations	22
Hardware and Configuration Notes.....	22
Required Tools and Versions.....	22
Configuration Steps.....	23

Abstract

The *Medical Journey Log: A Family Care Tracker* is a lightweight, personal-use application designed to help families document and manage the medical journey of a loved one undergoing long-term treatment. The system provides a simple interface for logging symptoms, appointments, journal reflections, and generating summary reports. Built with ASP.NET Razor Pages, C#, and MySQL, the application is tailored for use by a single caregiver account, which may be shared within the family. This Milestone 3 document outlines the final architecture plan, translating the requirements gathered in earlier phases into a detailed design ready for development in the upcoming phase.

Final Technical Requirements

The *Medical Journey Log: A Family Care Tracker* application will meet the following technical requirements:

System Functional Requirements

- Users can log, view, edit, and delete entries (symptoms, appointments, reflections)
- Each entry is categorized and searchable by type or date
- The system can generate PDF reports on-demand (e.g., symptom summary)
- A clean, accessible interface supports quick data input and filtering

System Architecture

- Single authenticated caregiver account (shared use by parents/caregivers)
- Local storage only — no multi-user accounts, cloud sync, or admin panel
- All user data is stored in a structured MySQL database
- Focus is on simplicity and reliability in a local environment

Software Requirements

Tool / Tech	Purpose
ASP.NET Razor Pages	Frontend web UI framework
C# (.NET Core)	Backend logic and routing
MySQL	Data storage and relational schema
MySQL Workbench	Database modeling and admin
Visual Studio for Mac	Integrated development environment
GitHub	Version control and project backup
Draw.io / Lucidchart	Diagramming system architecture
macOS (2018 MacBook Pro)	Development machine

Hardware Requirements

- 2018 MacBook Pro (Intel-based, 16 GB RAM)
- macOS Catalina (or later)
- Local instance of MySQL server

Design Planning Summary

The *Medical Journey Log: A Family Care Tracker* was created in response to a real-world need for better organization and documentation during long-term medical care. Families navigating complex medical conditions often struggle to track symptoms, log appointments, and reflect on emotional and treatment milestones in one place. This application is designed to offer a centralized, user-friendly platform for caregivers to record and access that information quickly.

The project continues to evolve based on instructor feedback and practical design considerations. As clarified in Milestone 2, the system is intended for a **single authenticated caregiver account**, with the understanding that multiple family members may share access under that single account for collaborative caregiving.

The planned design focuses on simplicity, local usability, and clarity. Features like symptom logging, appointment tracking, journaling, and PDF report generation have been scoped to ensure the solution remains lightweight and manageable, especially considering the developer's work and family obligations. The system is built using ASP.NET Razor Pages and C# for the frontend/backend logic, with MySQL as the data layer. All development is conducted on a macOS system using Visual Studio for Mac.

This design phase will finalize the technical architecture, data model, and user interface structure to guide implementation in the next milestone.

Overview of Design Concepts

The *Medical Journey Log* is designed to be intuitive, calm, and easy to navigate — particularly for caregivers who are often juggling emotional stress with logistical demands. The user interface prioritizes readability, minimalism, and speed of entry. All features are organized into clear, functional screens with simple navigation.

Main Navigation Flow

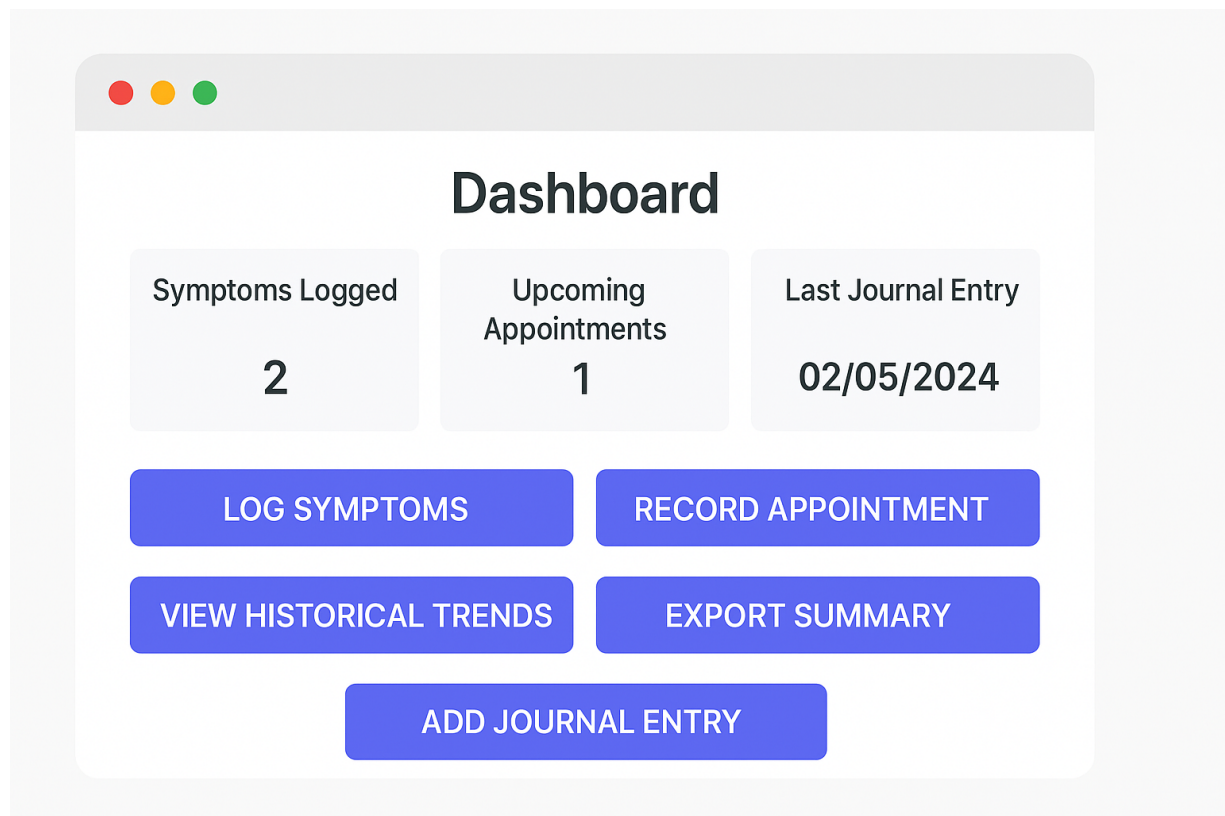
1. **Dashboard**
 - Displays a summary of recent activity: symptom logs, appointments, and latest journal entry.
2. **Log Symptom**
 - Allows entry of a new symptom using a dropdown (symptom type), text notes, and a date picker.
3. **Appointment Manager**
 - Users can add/edit/delete medical appointments via a calendar-style interface.
4. **Journal Entry**
 - Caregivers can reflect on the day using a large text box with a date stamp.
5. **Reports**
 - Provides exportable symptom summary reports in PDF format, filtered by date range.

User Interface Mockups

The following wireframes were originally created for Milestone 2 and continue to represent the planned user interface in Milestone 3. These designs follow a consistent macOS-style window format with a clean, user-friendly layout, using bold headings and large, accessible buttons.

1. Dashboard

- Displays total symptoms logged, upcoming appointments, and latest journal entry
- Quick access to all major actions (log symptoms, add appointment, journal entry, export)



2. Log Symptom

- Dropdown to select symptom type
- Text box for notes
- Date picker for entry date

A screenshot of a web application window titled "Log Symptom". The window has a light gray title bar with three colored window control buttons (red, yellow, green) on the left. The main content area is white and contains the following elements: a title "Log Symptom" in bold black font; a label "Symptom Type" above a dropdown menu with the text "Select..." and a downward arrow; a label "Notes" above a large, empty text input box; a label "Date" above a date input field showing the placeholder "mm/dd/yyyy" and a calendar icon; and a large blue button with the text "SUBMIT" in white capital letters at the bottom.

Log Symptom

Symptom Type

Select... ▼

Notes

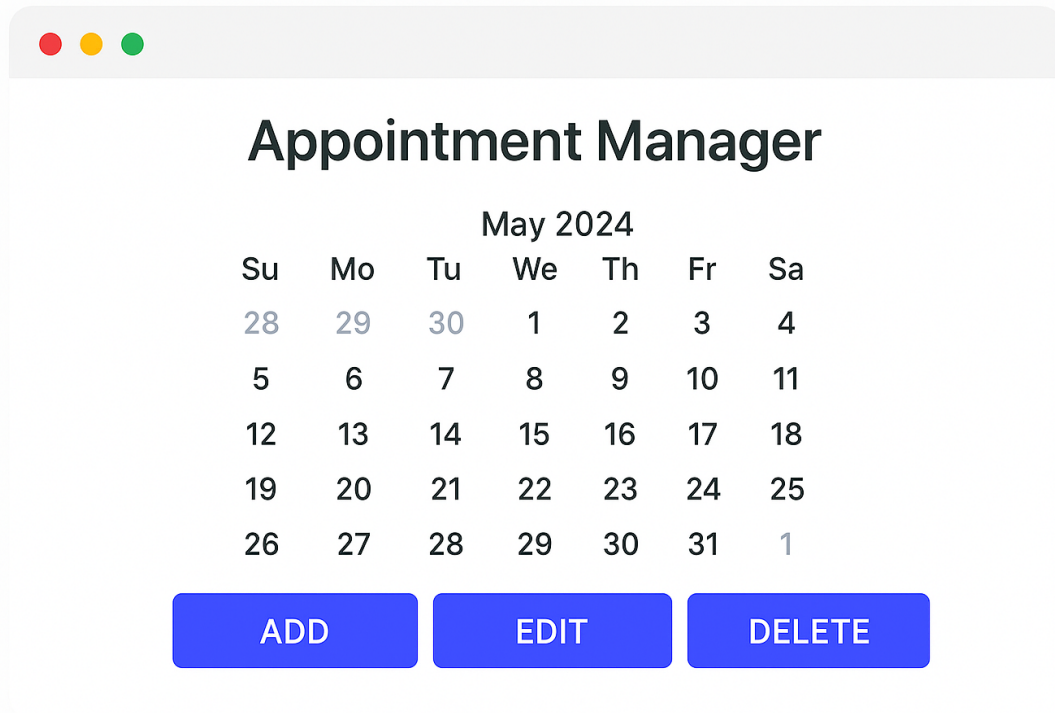
Date

mm/dd/yyyy 📅

SUBMIT

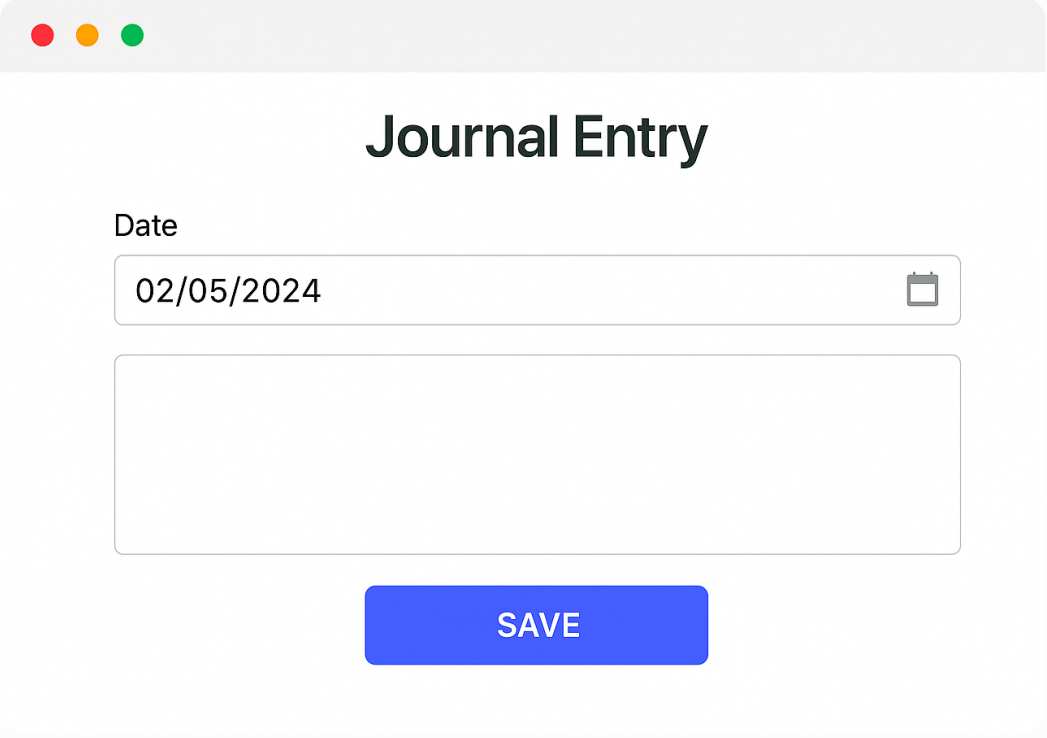
3. Appointment Manager

- Monthly calendar view with buttons to add, edit, or delete appointments



4. Journal Entry

- Date picker
- Large text field for reflection or journal content
- Save button

A mockup of a 'Journal Entry' form. It features a title 'Journal Entry' at the top, followed by a 'Date' label and a date input field containing '02/05/2024' with a calendar icon. Below the date field is a large, empty text area for journal content. At the bottom is a blue 'SAVE' button.

Journal Entry

Date

02/05/2024

SAVE

5. Reports

- Date range selector and optional family member filter
- Export button to generate a PDF report

The wireframe shows a window titled "Reports". Inside the window, there are two input fields: "Date Range" with a placeholder "mm/dd/yyyy - mm/dd/yyyy" and "Member Name" with a dropdown menu showing "Select..." and a downward arrow. Below these fields is a large blue button labeled "EXPORT PDF".

These wireframes guide the layout and interaction flow for the application's frontend and will inform the Razor Page structure in the development phase.

Pseudocode Example: Log Symptom

pgsql

CopyEdit

FUNCTION LogSymptom()

 INPUT: symptomType, notes, selectedDate

 IF symptomType IS EMPTY OR selectedDate IS EMPTY

 DISPLAY "Required fields missing"

 RETURN

 END IF

 CREATE newSymptomEntry WITH (symptomType, notes, selectedDate)

 SAVE newSymptomEntry TO MySQL

 DISPLAY "Entry saved successfully"

END FUNCTION

Design Priorities

- Form-based layout for fast input
- Categorization for better filtering
- Local storage to reduce complexity
- PDF export for medical appointments or recordkeeping

Detailed Solution Architecture

This section outlines the full technical design of the *Medical Journey Log* application. It provides all architecture details required to begin development, including data models, interface specifications, and process flow.

System Architecture Overview

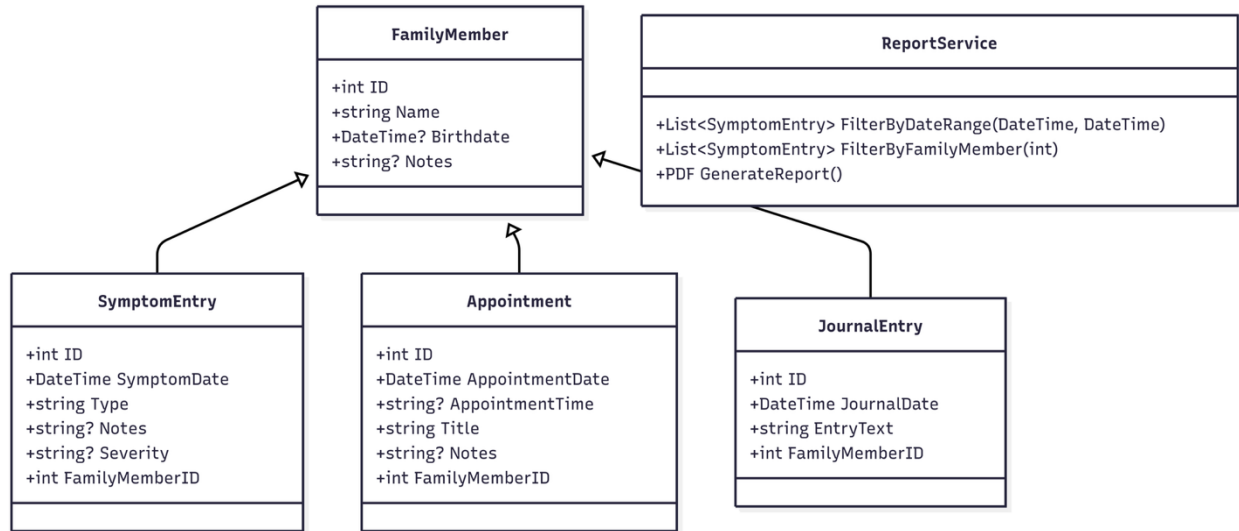
The system architecture is organized into clear, logical layers:

- **Frontend (UI Layer):** Built using ASP.NET Razor Pages, this layer provides form-based interfaces for logging symptoms, appointments, journal entries, and generating reports.
- **Application Logic Layer:** Implemented in C#, this handles routing, validation, and business rules.
- **Data Access Layer:** Interfaces directly with a MySQL database to manage CRUD operations for each data type.
- **Database Layer:** A MySQL database stores all persistent records securely and in structured tables.

The system is designed for local execution on macOS and does not rely on cloud infrastructure or external authentication services.

Object and Data Element Definitions

This section describes the core C# classes used in the application. Each class aligns closely with a corresponding database table and is responsible for encapsulating the properties and behavior related to medical data tracking. These objects represent the business logic layer in the application, handling input validation, data mapping, and functional interactions.



Class: FamilyMember

Represents a person (e.g., child) whose medical journey is being tracked.

Properties:

- int ID
- string Name
- DateTime? Birthdate
- string? Notes

Responsibilities:

- Stores identifying and contextual info for the individual
- Links to entries across the system (symptoms, appointments, journals)

Class: SymptomEntry

Captures daily health-related observations.

Properties:

- int ID
- DateTime SymptomDate
- string Type
- string? Notes
- string? Severity
- int FamilyMemberID

Responsibilities:

- Records symptom data for a family member
- Used in trend analysis and report generation

Class: Appointment

Tracks upcoming and past medical appointments.

Properties:

- int ID
- DateTime AppointmentDate
- string? AppointmentTime
- string Title
- string? Notes
- int FamilyMemberID

Responsibilities:

- Manages scheduling information for each family member
- Provides structured appointment logs and reminders

Class: JournalEntry

Allows caregivers to reflect or note events in narrative form.

Properties:

- int ID
- DateTime JournalDate
- string EntryText
- int FamilyMemberID

Responsibilities:

- Supports free-form journaling or contextual notes
- Useful for emotional tracking, side observations, or milestones

Class: ReportService

Handles data aggregation and export functionality.

Methods:

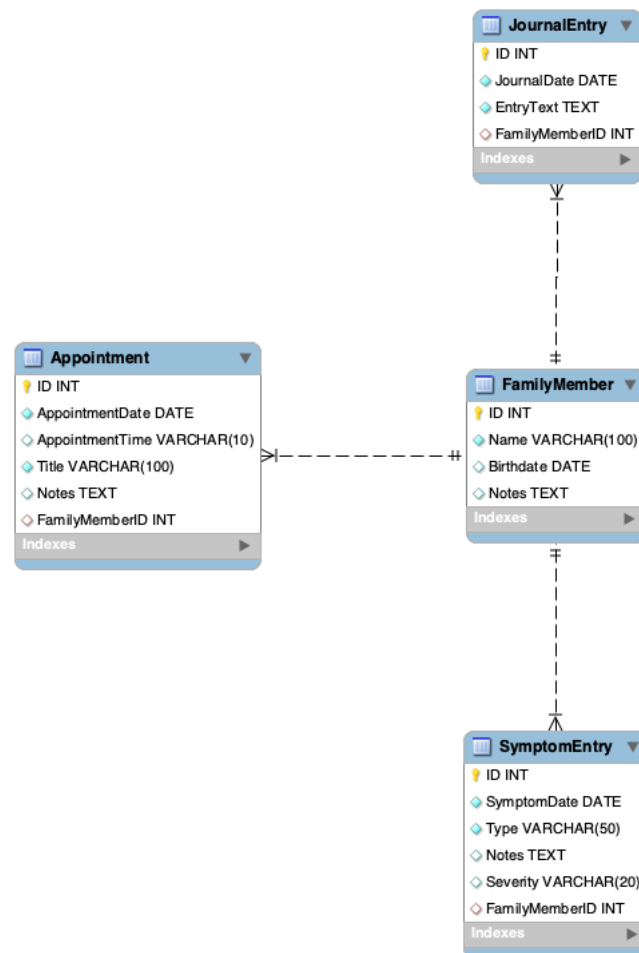
- List<SymptomEntry> FilterByDateRange(DateTime start, DateTime end)
- List<SymptomEntry> FilterByFamilyMember(int familyMemberID)
- PDF GenerateReport(...)

Responsibilities:

- Prepares filtered data for review or export
- Supports future functionality such as generating CSV/PDF files for clinics or personal logs

Entity-Relationship Diagram / Database Schema

The database is designed using MySQL and consists of structured tables representing key record types in the application. The addition of a FamilyMember table allows for entries to be associated with multiple children or individuals under a single caregiver account. This enables the caregiver to track and organize medical information per family member without complicating the user experience.



Planned Tables:

- FamilyMember
- SymptomEntry
- Appointment
- JournalEntry

Table Definitions

FamilyMember

Field	Type	Key	Description
ID	INT	PK	Unique identifier
Name	VARCHAR(100)		Full name of the family member
Birthdate	DATE		Optional birthdate
Notes	TEXT		Any relevant notes or background

SymptomEntry

Field	Type	Key	Description
ID	INT	PK	Auto-increment symptom record ID
SymptomDate	DATE		Date symptom occurred
Type	VARCHAR(50)		Symptom category/type
Notes	TEXT		Optional description or context
Severity	VARCHAR(20)		Optional severity label
FamilyMemberID	INT	FK	Links to FamilyMember(ID)

Appointment

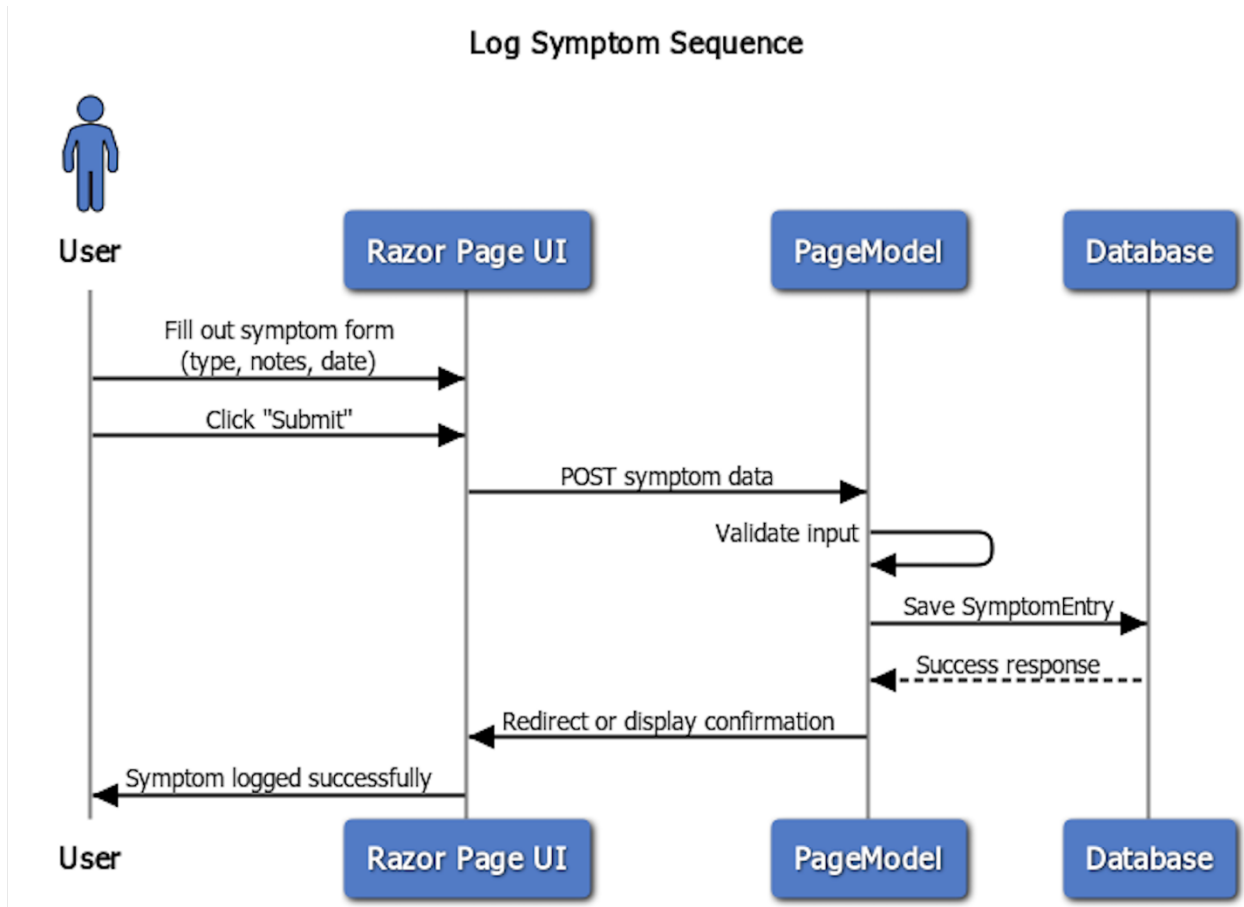
Field	Type	Key	Description
ID	INT	PK	Appointment ID
AppointmentDate	DATE		Scheduled date
AppointmentTime	VARCHAR(10)		Scheduled time
Title	VARCHAR(100)		Short description of the appointment
Notes	TEXT		Additional appointment details
FamilyMemberID	INT	FK	Links to FamilyMember(ID)

JournalEntry

Field	Type	Key	Description
ID	INT	PK	Entry ID
JournalDate	DATE		Date the entry was written
EntryText	TEXT		Reflection, observation, or note
FamilyMemberID	INT	FK	Links to FamilyMember(ID)

Sequence Diagram

The following diagram illustrates how the system handles a typical interaction, such as logging a new symptom.



Actors:

- User
- Razor Page UI
- PageModel / Controller
- Database

This sequence outlines the typical flow for a user logging a symptom in the Medical Journey Log application:

1. Request (User → Razor Page UI):

The user fills out the symptom logging form, including fields such as type, notes, and date. Upon clicking "Submit," the data is sent via a POST request to the backend.

2. **Validation (Razor Page UI → PageModel):**

The PageModel receives the form data and performs input validation, checking for required fields, proper formatting, and any business rules.

3. **Save (PageModel → Database):**

If validation is successful, the system proceeds to persist the data by creating a new SymptomEntry record in the database.

4. **Confirmation (Database → User):**

After the database confirms successful storage, a response is passed back through the PageModel and UI to the user. The UI either redirects to a confirmation page or displays a success message such as “Symptom logged successfully.”

Final UI Screen Specifications

Each of the following screens is represented visually in the wireframes and supports the core functionality of the app:

Dashboard

- **Summary Tiles:**
 - Symptom Entries Today | Total Appointments | Journal Notes This Week
 - Tooltips: Hover to see counts based on the current user or family member filter.
- **Navigation Buttons:**
 - “Log Symptom” – Tooltip: *Quickly add a new symptom for any family member*
 - “Add Appointment” – Tooltip: *Jump to calendar to schedule upcoming visits*
 - “Write Journal” – Tooltip: *Record thoughts, notes, or updates for today*

Log Symptom

- **Fields:**
 - Symptom Type (Dropdown): Required
 - Tooltip: *Select the primary symptom observed or experienced*
 - Notes (TextArea): Optional
 - Tooltip: *Describe the symptom, context, or related behavior*
 - Severity (Dropdown, Optional): e.g., Mild, Moderate, Severe
 - Date (Date Picker): Defaults to today; required
- **Validation:**
 - Symptom Type and Date must be selected before saving

- Optional notes limited to 500 characters

Appointment Manager

- **Calendar View:**
 - Displays appointments per family member with hover popups for details
- **Add/Edit Modal:**
 - Title (TextField): Required
 - Date (Date Picker): Required
 - Time (Time Picker or TextField): Optional
 - Notes (TextArea): Optional
 - Family Member (Dropdown): Required
- **Buttons:**
 - “Add” | “Edit” | “Delete” — All with confirmation modals
 - Tooltips: Clarify function and whether changes are saved automatically

Journal Entry

- **Fields:**
 - Date (Auto-filled, editable)
 - Entry Text (Multi-line TextArea): Required
 - Tooltip: *Write notes about the day, behavior, mood, etc.*
- **Validation:**
 - Entry Text must not be blank
 - Max length: 1000 characters
- **Save Button:**
 - Shows confirmation banner on success

Reports

- **Filters:**
 - Date Range (From/To Pickers): Required
 - Family Member (Dropdown): Optional
- **Export Button:**
 - Tooltip: *Download a summary report as PDF*
 - Disabled until valid date range is selected
- **Validation:**
 - Ensure "From Date" is before "To Date"
 - Require at least one record to be found before export is enabled

Security Considerations

The system is designed for personal, local use. Security measures are intentionally lightweight to reflect the offline, single-user nature of the application.

- **Access Control**
 - A single caregiver login provides access to all features.
 - The account can be shared among trusted family members if needed.
 - No internet-based logins or external authentication systems are used, reducing exposure to online threats.
- **Data Storage**
 - All application data is stored locally in a MySQL database instance on the user's computer.
 - No data is transmitted over the internet, APIs, or cloud services.
 - The system assumes a trusted user environment (e.g., home computer or secure laptop).
- **Encryption**
 - Encryption at rest is not currently implemented.
 - This is acceptable for the offline model but may be reconsidered in future versions that support remote access or syncing.
 - Application-level passwords and MySQL user permissions can be configured locally as an added measure.

Hardware and Configuration Notes

This application is developed and tested on a 2018 MacBook Pro and is intended for offline, local use in a secure home environment.

Required Tools and Versions

- **macOS:** 10.15 Catalina or later
- **IDE:** Visual Studio for Mac (Community Edition)
- **Framework:** .NET Core SDK (latest supported version at time of development)
- **Database:** MySQL Community Server (8.x recommended)
- **Version Control:** GitHub (local repository used for backup and tracking; no CI/CD or remote hosting)

Configuration Steps

1. Install Prerequisites

- Download and install Visual Studio for Mac with support for ASP.NET Core and Razor Pages.
- Install the .NET Core SDK from the official Microsoft site.
- Install MySQL Community Server and MySQL Workbench for database administration.

2. Database Setup

- Create a new local MySQL schema (e.g., familycaretracker).
- Run the provided SQL schema script (if available) or allow the application to initialize tables on first use.

3. Environment Variables

Add the following environment variables to your development machine or set them in the Visual Studio launchSettings.json:

- DB_HOST=localhost
- DB_PORT=3306
- DB_USER=root *(or your chosen username)*
- DB_PASSWORD=yourpassword
- DB_NAME=familycaretracker

4. Optional Settings

- Configure .gitignore to exclude appsettings.json or sensitive .env files if committing code to GitHub.
- Use dotnet user-secrets if environment variable management is preferred within development.